

A Boundary-Driven Architecture for Grounded, Auditable LLM Support: A Case Study in Electricity Billing Assistance

Kaustubha Venkata Eluri
Northeastern University

Abstract—Customer-facing billing assistants built on large language models (LLMs) face a specific and consequential failure mode: confidently fabricating numbers, rates, or policy clauses that do not exist. This paper documents the design of a prototype electricity billing support system that combines four deliberately simple patterns to mitigate this risk: MCP-style tool calling for deterministic data access, TF-IDF retrieval-augmented generation (RAG) over a small policy corpus (with no embeddings or vector database), keyword-based intent routing in place of an LLM-based router, and grounded LLM synthesis with an explicit, parsed confidence label. We describe the system’s request-flow architecture, the cost/auditability rationale behind each design choice as documented in the project’s architecture decision records (ADRs), and a concrete case study in which the underlying LLM vendor was migrated from Anthropic’s Claude to Google’s Gemini as a single-file change. We report only the testing and evaluation that is actually documented for the system — which is informal, manual, and code-path-coverage-oriented rather than benchmarked — and we discuss the system’s documented limitations and roadmap honestly rather than inventing performance figures.

Index Terms—retrieval-augmented generation, intent routing, tool-augmented LLMs, grounded synthesis, hallucination mitigation, software architecture

I. INTRODUCTION

Large language models are increasingly proposed as the front end for customer-facing support systems, including utility billing support. This is an attractive proposition for utilities because natural-language interfaces can absorb free-form questions (“why is my bill higher than last month?”) that do not map cleanly onto a fixed menu of self-service options. However, billing is a domain where factual precision is not optional: a dollar figure, a kWh usage number, or a stated peak-pricing rate that the model invents rather than retrieves from a real record is not a stylistic flaw but a trust-destroying and potentially regulatorily significant error.

This risk is compounded by a practical constraint: intent routing and retrieval infrastructure themselves have cost and latency budgets. A naive design might route every incoming query through an LLM classifier and retrieve context via a dense embedding index and vector database. For a small, well-scoped support domain, this adds LLM-classification latency and cost to every single request, and it introduces retrieval and routing behavior that is difficult to audit when something goes wrong, because embedding similarity and LLM classification decisions are comparatively opaque to a support engineer trying to explain a misrouted query.

This paper documents a prototype system — an electricity billing assistant — that takes the opposite approach on both axes. Rather than assembling a single *sophisticated* pipeline, the system co-locates four separate, individually simple patterns and relies on the explicit *contract* between them to produce a system that is inexpensive, low-latency, and auditable end to end:

- 1) MCP-style tool calling for deterministic, structured access to billing data;
- 2) TF-IDF retrieval over a small, static policy document corpus, with no embedding model or vector database;
- 3) keyword-based intent routing rather than an LLM-based classifier;
- 4) grounded LLM synthesis, in which the model is only shown tool output and retrieved policy text and must append a machine-parseable confidence label to every answer.

The system also serves as a case study in boundary-driven software architecture: over the project’s history, the LLM vendor was migrated from Anthropic’s Claude to Google’s Gemini, and because the LLM call was isolated behind a single function in one file, the migration touched only that file. We treat this migration as direct evidence for a claim that is often made about clean architectural boundaries but rarely demonstrated concretely.

Consistent with the scope of the source material available for this paper, we do not report benchmarked accuracy or latency numbers that are not documented for this system; where the project’s own documentation states that no such evaluation exists, we say so rather than estimating or inventing figures.

II. RELATED WORK

A. Retrieval-Augmented Generation

Retrieval-augmented generation (RAG) systems address LLM factual unreliability by conditioning generation on retrieved external documents rather than relying solely on parametric knowledge. The dominant implementation pattern in recent industry and research practice retrieves documents via dense vector embeddings and approximate nearest-neighbor search over a vector database. This is effective at scale and captures semantic similarity beyond lexical overlap, but it introduces infrastructure (an embedding model, an index,

a vector store) whose cost is justified primarily when the underlying corpus is large or semantically diverse. Classical sparse retrieval methods based on term-frequency statistics, such as TF-IDF and its relatives, predate the embedding-based approach and remain a reasonable choice when a corpus is small, static, and largely non-synonymous in its vocabulary.

B. Intent Classification and Routing

Task-oriented dialogue and support systems typically include an intent-routing stage that decides which downstream handler — a database lookup, a knowledge-base search, a scripted flow, or free-form generation — should service a given query. Two broad families exist: statistical/learned classifiers (increasingly LLM-based, given a query and a small set of candidate intents) and rule-based or keyword-based classifiers. Learned and LLM-based routers can generalize to novel phrasings but are comparatively expensive to run on every request and harder to explain when they misclassify. Rule-based routers are cheaper and more transparent but require ongoing maintenance of the rules or keyword sets as user vocabulary drifts.

C. Tool-Augmented LLMs

A separate line of work equips LLMs with the ability to invoke external functions or “tools” — often standardized through protocols resembling the Model Context Protocol (MCP) — to fetch structured, deterministic data (e.g., a database record) rather than relying on the model to recall or guess such data from its own training. Combining tool calling with retrieval and grounded generation is a natural extension: tools answer “what are the facts,” retrieval answers “what is the relevant policy context,” and the LLM is restricted to synthesizing an answer strictly from what the other two components provide.

The system described in this paper sits at the intersection of these three threads, but with a deliberate, documented preference for the simpler member of each pair (keyword routing over LLM routing; TF-IDF over embeddings) justified specifically by the small, well-bounded scope of the target domain.

III. SYSTEM ARCHITECTURE

The system exposes a single logical operation — answering a natural-language billing question for a given customer — through two presentation surfaces: a FastAPI REST endpoint (POST /query) and a Streamlit chat UI. Both surfaces share one underlying pipeline: `router.py` → `tools.py` / `rag.py` → `llm.py`. The project’s architecture documentation frames this explicitly as “two surfaces, one core,” so that a bug or fix in routing or grounding surfaces identically, and is fixed identically, on both the REST and chat presentation layers.

A. Request Flow

A query such as “Why is my bill higher? C003” flows through the pipeline as follows, and Figure 1 shows a representative resulting response in mode `both`:

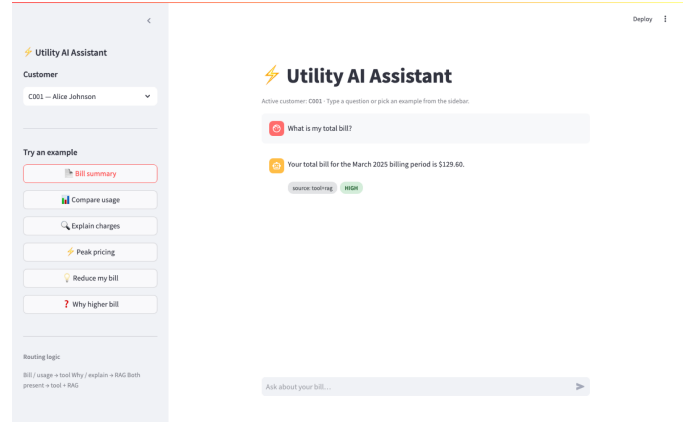


Fig. 1. A representative `tool+rag` response in the chat UI: the tool-fetched billing figures and the retrieved policy explanation are synthesized into a single grounded answer, with the self-reported confidence label surfaced alongside the response.

- 1) **Routing.** `router.py` scores the query’s tokens against two fixed keyword sets — one associated with tool-answerable queries (e.g., “bill,” “usage,” “kwh,” “charge,” “breakdown”) and one associated with policy/explanatory queries (e.g., “why,” “explain,” “policy,” “peak,” “reduce”). If both scores are positive the query is routed to mode `both`; if only the tool score is positive, mode `tool`; if only the RAG score is positive, mode `rag`; if neither is positive, the router defaults to `rag` rather than guessing a tool call. A customer identifier (pattern `C\d{3}`, e.g., `C003`) is extracted from the query text via regular expression, or accepted explicitly as a `customer_id` field.
- 2) **Tool execution.** If the mode includes `tool`, one of three MCP-style functions in `tools.py` — `get_bill`, `compare_usage`, or `explain_charges` — reads structured mock customer data from `data/billing_data.json` and returns structured JSON (e.g., current usage 1240 kWh, previous usage 890 kWh, trend `significantly_higher`).
- 3) **Retrieval.** If the mode includes `rag`, `rag.py` retrieves the top- k (with $k = 2$) most relevant sections of a small policy corpus (`data/docs.txt`, chunked at startup by `SECTION: headers`) using a scikit-learn TF-IDF vectorizer and cosine similarity.
- 4) **Grounded synthesis.** `llm.py` assembles a context block consisting of a `[TOOL DATA]` section (the tool’s JSON output or an error message) and a `[POLICY DOCS]` section (the retrieved chunks with their titles). This context, together with a system prompt instructing the model to answer *only* from the supplied context and to quote exact dollar/kWh figures rather than inventing them, is sent to `gemini-2.5-flash`. The model is further instructed to end every response with a line of the form `CONFIDENCE: HIGH|MEDIUM|LOW`, which a deterministic parser (`_parse_confidence`) extracts from the tail of the response, defaulting to `LOW` if no

label is found.

- 5) **Response.** The API returns a JSON object with an answer string, a source tag (tool, rag, or tool+rag), and a confidence field (HIGH/MEDIUM/LOW).

B. Four Deliberate Design Patterns as Cost/Auditability Tradeoffs

Each of the four patterns combined in this system was chosen, per the project’s documentation, specifically *instead of* a more elaborate alternative, on cost and auditability grounds rather than because the simpler alternative is unconditionally superior.

a) *Tool calling with typed JSON outputs.*: Billing facts (current bill total, usage figures, itemized charges) are never left to the LLM to recall or infer; they are fetched deterministically from a structured data source and returned as typed JSON. This removes an entire class of numeric hallucination at the source, at the cost of requiring an explicit, maintained mapping from intents to tool functions.

b) *TF-IDF retrieval instead of embeddings.*: Because the target policy corpus is small (seven sections, on the order of a few hundred tokens in total), the system uses a locally computed TF-IDF index with cosine similarity rather than an embedding model and vector database. This avoids external embedding API calls (both at index-build time and at query time), avoids operating a vector store, and runs in milliseconds with no network dependency. The documented tradeoff is that TF-IDF is a bag-of-words method and will miss synonymy and paraphrase that an embedding-based retriever would catch; the project’s own decision record states this is an acceptable tradeoff *at this scale* and identifies a concrete trigger for revisiting it (corpus growth past roughly 500 chunks), at which point the retrieval *contract* — top-*k* by cosine similarity — would not need to change, only its implementation.

c) *Keyword-based routing instead of LLM-based routing.*: Routing occurs on every single incoming query, so its cost structure matters more than that of a one-off classification. A keyword scorer runs in microseconds at zero marginal cost, versus an LLM router’s tens-to-hundreds of milliseconds and per-call cost. The more consequential argument made in the system’s documentation is auditability: when a keyword router misroutes a query, an engineer can read the exact keyword scores that produced the decision, whereas an LLM router’s decision is comparatively opaque. The documented cost of this choice is that the keyword sets require ongoing maintenance as user vocabulary drifts, and queries containing no recognized keywords receive only a generic default; this is mitigated, not eliminated, by defaulting ambiguous or unmatched queries to the retrieval path (rag) rather than risking an incorrect tool invocation with a possibly-wrong customer ID.

d) *Grounded synthesis with explicit confidence labeling.*: The system prompt explicitly forbids the model from inventing numbers, rates, or policies, and structurally the model is never given access to the raw customer database — only to the tool’s already-validated output and the retrieved policy text.

On top of this, every response is required to end with a self-reported CONFIDENCE label that is parsed out programmatically and surfaced as a first-class, color-coded field in the UI and API response, rather than being buried in prose. The documented rationale is that most LLM-based applications treat every response as equally confident by default; making a LOW-confidence, “insufficient context” outcome reachable and visible is treated as a deliberate design goal rather than a fallback to be hidden.

IV. DESIGN DECISIONS

The system’s decision records (docs/decisions.md) document ten accepted architecture decisions (ADR-001 through ADR-010). Several are elaborated above as the four core patterns (ADR-002 keyword router; ADR-003 TF-IDF retrieval; ADR-004 grounded synthesis; ADR-005 confidence labeling); this section highlights the remaining decisions, in particular the vendor-migration story.

A. ADR-001: Four Patterns Co-located, Not Demonstrated in Isolation

The project deliberately combines all four patterns in one small codebase rather than showcasing each independently. The stated rationale is that the value is not in any individual pattern but in the *contract* between them: the router decides which source(s) to consult, tools produce structured data, RAG produces grounded text context, and the LLM is restricted to synthesizing only from what the other components hand it. Demonstrating that contract, not any one component, is described as the point of the project.

B. ADR-006: Mock Mode Covers Every Code Path Except the LLM Call

Without a GEMINI_API_KEY set, llm.py falls back to a deterministic, template-based _mock_generate() function keyed off the tool and RAG output, rather than a random or empty response. Routing, tool execution, retrieval, response shape, and confidence-label assignment all execute identically to the live path; only the final LLM synthesis call is skipped. This is presented as a practical necessity so that neither developers nor CI need an API key to exercise the full application, and the mock path’s confidence rules (tool success → HIGH; tool error → LOW; RAG-only with top cosine score above 0.2 → HIGH, else MEDIUM; no tool and no RAG match, e.g. a greeting → HIGH) are designed to mirror what a live LLM call would plausibly produce.

C. ADR-007: The LLM Vendor Boundary — A One-File Migration

The most notable decision record for the purposes of this paper is ADR-007, titled “Single app/llm.py is the LLM-vendor boundary.” The system’s llm.py module was originally built against the Anthropic API using Claude. A subsequent commit migrated the module to Google’s google-generativeai library and the gemini-2.5-flash model. According to the decision

record, this migration changed *only* `app/llm.py`: routing logic, the tool functions, the RAG index and retrieval logic, the response JSON shape, and the confidence-parsing logic were all left untouched.

The architectural reason this was possible is that every other component in the system interacts with the LLM exclusively through a single function, `generate_answer`, which acts as a protocol-style boundary: callers pass in an assembled context and receive back a text answer plus a confidence label, with no caller-side knowledge of which vendor, model, or SDK sits behind that function. Swapping the vendor is therefore a matter of reimplementing the internals of one function in one file, not of touching every call site that happens to talk to an LLM. The decision record explicitly frames this as evidence for a general architectural claim: that isolating an external, volatile dependency (an LLM vendor API, in this case) behind a narrow, stable internal interface pays for itself the moment that dependency needs to change, which is a recurring and often underestimated cost in LLM-integrated systems given how quickly the vendor landscape and pricing/performance tradeoffs shift. The same record notes that if the system later needed to support multiple providers concurrently (for example, a fallback dispatch strategy), `llm.py` would need to evolve from a single implementation into a registry, but that this remains a localized change to the same file/boundary rather than a system-wide refactor.

D. ADR-008 through ADR-010: Supporting Decisions

ADR-008 documents the shared-core, two-surface design (FastAPI and Streamlit) discussed in Section III. ADR-009 documents the choice of a simple `C\d{3}` regular expression for customer ID extraction from free text, with an explicit note that a production system would need to generalize this pattern. ADR-010 documents secret management via a git-ignored `.env` file loaded through `python-dotenv`, with `.env.example` committed as a template, chosen as the simplest pattern that works uniformly across local development and CI (where CI sets the API key as a workflow environment variable rather than a file).

V. EVALUATION

The project’s own documentation does not describe a formal, quantitative evaluation of this system, and this paper does not report one. No benchmark suite, held-out query set, latency measurement, or accuracy/precision metric is documented in the project’s README or `docs/` directory. We state this directly rather than substituting invented numbers.

What *is* documented, under the project’s stated “Quality Gates,” is a set of qualitative, code-path-level checks that the system is exercised against:

- The FastAPI application (`uvicorn app.main:app`) starts cleanly on Python 3.10+, and the Streamlit UI (`streamlit run ui.py`) launches without error.
- All three tools (`get_bill`, `compare_usage`, `explain_charges`) return structured JSON for valid

mock customer IDs and structured error responses for invalid ones.

- RAG retrieval returns top-*k* results ranked by cosine similarity, with a documented score-based confidence floor in mock mode (HIGH if the top score exceeds 0.2, otherwise MEDIUM).
- The LLM grounding rules are stated explicitly in the system prompt, and exact dollar/kWh figures are expected to be quoted verbatim from `[TOOL DATA]` rather than restated approximately.
- The `CONFIDENCE: HIGH/MEDIUM/LOW` label is parsed deterministically from the tail of each response.
- Secrets (`.env`) are gitignored and never committed.
- With no `GEMINI_API_KEY` set, the mock-mode code path exercises every branch of the application except the live LLM call, meaning the UI and API are fully driveable end to end without external dependencies.

These gates constitute manual/structural verification of code-path coverage and output shape, not a measurement of answer correctness, hallucination rate, routing accuracy, or latency against ground truth. In particular, the project’s own limitations documentation (Section VI) explicitly states that no hallucination regression tests exist and that confidence labels are self-reported by the model and uncalibrated — i.e., a HIGH label is not claimed, nor shown, to correspond to any measured empirical accuracy rate. We treat the absence of such an evaluation as itself a finding worth reporting honestly, and the project’s own roadmap (Section VII) identifies closing this gap as future work rather than as already accomplished.

VI. LIMITATIONS

The project’s `docs/limitations.md` enumerates limitations across several categories, summarized here.

Scope. The system is a single-domain (electricity billing) prototype operating over mock data — four customers and seven policy sections — and is explicitly not a production billing system. The `POST /query` endpoint has no authentication, and no query or answer history is persisted beyond Streamlit’s in-session state.

Routing. The fixed keyword sets used for intent routing are vulnerable to vocabulary drift: new phrasings, synonyms, or typos not present in the keyword lists will route incorrectly, and a query such as “how much do I owe?” would not be recognized as a tool query because it contains none of the configured tool keywords. Routing is also stateless across turns; a multi-turn follow-up such as “tell me more” after a billing answer does not route back to the tool that produced that answer.

RAG. TF-IDF retrieval is fundamentally bag-of-words and will miss semantically related but lexically dissimilar phrasing; there is no re-ranking or diversification (e.g., maximal marginal relevance) beyond raw cosine similarity ranking; and the system does not surface which specific document sections were used to ground a given answer (no “per Section X” citation).

LLM grounding. The “do not invent numbers or policies” instruction is enforced only as a soft constraint in the system prompt; there is no output-side validation that rejects or flags a response containing a number not present in [TOOL DATA], and no hallucination regression test suite exists.

Confidence calibration. Confidence labels are self-reported by the LLM and are explicitly documented as uncalibrated — a HIGH label carries no claim about actual empirical accuracy — and there is no threshold-based escalation (e.g., to a human agent or a different model) when a LOW-confidence response is produced.

Performance and operations. The system has no response caching, no request batching, and no streaming (Gemini’s streaming API is not used; the full response is awaited). Logging is unstructured (`print()` and default FastAPI logs), there is no metrics export or latency instrumentation, no rate limiting, and no formal availability target (error budget/SLO).

VII. CONCLUSION AND FUTURE WORK

This paper documented a small prototype electricity billing assistant that combines four individually simple design patterns — deterministic tool calling, TF-IDF retrieval, keyword-based routing, and grounded synthesis with explicit confidence labeling — and argued, based on the system’s own architecture decision records, that the value of the design lies less in any single component and more in the explicit contract enforced between them: tools and retrieval supply facts and context, and the LLM is structurally restricted to synthesizing only from what those two components provide. We further examined a concrete instance of boundary-driven architecture paying off: a full LLM vendor migration from Anthropic’s Claude to Google’s Gemini that was confined to a single file because every other component depended on a single, narrow `generate_answer` interface rather than on any vendor-specific detail.

Consistent with the scope of this paper, we reported the system’s evaluation honestly: no formal accuracy, hallucination-rate, or latency benchmarking is documented, and the qualitative “quality gates” that do exist verify code-path coverage and output shape rather than correctness against ground truth.

The project’s own roadmap identifies concrete future work consistent with these limitations: output validation that rejects LLM responses containing numeric tokens absent from the supplied tool data; surfacing which policy sections were used to ground a given answer; a small hallucination regression test suite; a semantic-routing fallback (invoking a cheap LLM classifier only when the keyword router finds no match, with the classifier’s verdicts logged to grow the keyword set over time); streaming responses; authentication and rate limiting; persistence of customer data and conversation history; and, longer-term, a research effort to empirically calibrate the self-reported HIGH/MEDIUM/LOW confidence labels against human-judged correctness. The roadmap also identifies a concrete, corpus-size-based trigger (roughly 500 chunks) for revisiting the TF-IDF-versus-embeddings decision, while noting that the underlying retrieval contract — top- k by similarity

— would not need to change even if the implementation did. Taken together, these documented next steps suggest a system whose current simplicity is a deliberate, revisitable starting point rather than a permanent ceiling.